UNIVERSITY OF AGRICULTURAL SCIENCES, BENGALURU-560065

COLLEGE OF AGRICULTURE,GKVK

EAI 421
INTRODUCTION TO ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

PROJECT REPORT ON MANGO RIPENESS DETECTION USING AI AND ML

SUBMITTED BY:

Lokesh M K– AMB 2120,
Malappa Pujari – AMB 2128,
Mallikarjun – AMB 2129,
4th Year Bsc(Hons) Agriculture,
HOT Module-01.

SUBMITTED TO:

The course teacher,
Introduction to AI & ML in agriculture,
HOT Module – 01,
UASB, GKVK.

Acknowledgement

I would like to express my sincere gratitude to the professors from REVA University, Bangalore, for conducting Introduction to Artificial Intelligence and Machine Learning course and for sharing their valuable knowledge and expertise, which greatly contributed to the successful completion of this project. Their guidance provided us with a strong foundation in AI and ML concepts and helped us understand their practical applications in real-agricultural problems.

I am especially thankful to our course in-charge for her continuous support, encouragement, and coordination throughout the course and project work. Her guidance was instrumental in helping us complete this project effectively.

I also thank my university for organizing this learning opportunity and providing access to such a beneficial program. The course was highly useful in enhancing our laboratory and practical orientation, enabling us to gain hands-on experience in applying AI and machine learning techniques in agricultural projects.

INDEX

S.No	Particulars	Pg.No
1.	Introduction	1
2.	Problem statement & Scope of the Project	1-2
3.	Motivation	2-3
4	Objectives of the project	3
5.	Dataset Overview	3-4
6.	Methodology/Approach •Workflow diagram •Python Code	5-7 8-11
7.	Tools and Technologies used	12
8.	Results	13
9.	Analysis	14
10.	Conclusion	15
11.	References	16

Introduction

•Agriculture plays a vital role in India's economy, with fruit production being a major component. Mango, known as the “King of Fruits,” requires precise harvesting and sorting based on ripeness for better market value and reduced post-harvest losses.

•Traditionally, ripeness is assessed manually using color, texture, and smell, which is subjective, inconsistent, and time-consuming.

•This creates a need for an automated, reliable, and scalable solution.

•With advancements in Artificial Intelligence (AI) and Deep Learning, image-based classification techniques can be applied to detect ripeness levels accurately.

•Convolutional Neural Networks (CNNs), especially transfer learning models like EfficientNet, provide high accuracy in image classification tasks.

•This project focuses on building a deep learning model to classify mangoes into different ripeness stages using image data.

Problem Statement:

•Manual identification of mango ripeness is subjective and error-prone.
•Lack of consistency in grading affects market pricing and quality.
•Need for an automated, fast, and accurate ripeness detection system.
•Classification problem: categorize mango images into ripeness stages.
•Scope limited to image-based classification using deep learning.

Scope of this project:

• Develop an AI-based system to automatically classify mango ripeness stages from images
• Detect and categorize mangoes into classes such as Unripe, Semi-Ripe, and Ripe.
• Use CNN/deep learning techniques for image-based ripeness prediction.
• Reduce manual errors in traditional visual inspection methods.
• Support farmers, sellers, and supply chain managers in quality grading.
• Enable future integration into mobile apps, smart sorting machines, and real-time agricultural systems .
• Scalable for extension to other fruits like banana, apple, or tomato ripeness detection.

Motivation:

Why this problem is important?

•Mango ripeness directly affects fruit quality, taste, and market value
•Manual ripeness detection is inconsistent and prone to human error
•Incorrect grading can lead to financial losses and food wastage
•Automated detection ensures faster and more reliable classification

Practical applications in agriculture/farming

•Helps farmers identify the correct harvest time for better yield quality
•Assists sellers and exporters in sorting mangoes accurately for market standards
•Useful in packaging and storage units for automated grading systems
•Reduces labor dependency in large-scale fruit processing industries

Relevance to research, industry, and sustainable farming

•Promotes adoption of AI technologies in precision agriculture
•Supports smart farming practices through data-driven decision-making
•Encourages sustainable farming by minimizing waste and improving supply chain efficiency
•Creates opportunities for future research in fruit quality assessment and automated agricultural monitoring

Objectives of the Project:

Primary Objectives:

•Develop an image classification model for mango ripeness detection
•Classify mangoes into 3 categories (e.g., Unripe, Semi-ripe, Ripe)
•Use transfer learning for improved performance

Technical Objectives:

•Apply data preprocessing and augmentation
•Train CNN
•Evaluate model using accuracy metrics

Expected Outcomes:

•Accurate classification model
•Reduced human dependency
•Insight into deep learning applications in agriculture

Dataset Overview

Dataset Source:

The dataset is collected from a publicly available mango ripeness image dataset (Kaggle) and used for training the deep learning model.

Number of Records and Features:

•Total records: Mango image samples categorized into 3 classes

•Number of classes/features: 3 ripeness categories

1. Unripe
2. Semi-Ripe
3. Ripe

Types of Features:

•Categorical Feature: Ripeness class label (Unripe, Semi-Ripe, Ripe)

•Visual Crop Features: Color, texture, skin appearance, and ripeness patterns.

•Since this is image-based data, environmental and soil features are not included.

Dataset

	UNRIPE	RIPE	OVER-RIPE
SAMPLE			

Methodology / Approach:

1. Data Cleaning & Preprocessing

The mango image dataset contains three classes:

- Unripe
- Ripe
- Overripe

Images are organized into folder-wise labeled directories.

Preprocessing steps applied:

- Resizing all images to 224 × 224 pixels
- Normalization of pixel values (0–255 scaled to 0–1)
- Data augmentation:
- Rotation
- Zooming
- Horizontal flipping

Dataset split:

- 80% Training data
- 20% Validation data

2. Exploratory Data Analysis (EDA)

•Checked dataset structure and class distribution.
•Verified equal representation of all three ripeness classes.
•Visualized sample mango images from each category.
•Examined image dimensions and label consistency.

3. Model / Algorithm Selection: CNN

•Chosen model: Convolutional Neural Network using EfficientNetB0 architecture.
•Reason for selection:
•High performance in image classification tasks
•Efficient feature extraction from fruit images
•Suitable for transfer learning with smaller datasets

4. Model Training and Validation

Pre-trained EfficientNetB0 used as base model.

Added custom CNN layers:

- Global Average Pooling Layer
- Dense Layer (128 neurons, ReLU activation)
- Dropout Layer (0.3)
- Output Layer (Softmax activation for 3 classes)

Optimizer used: Adam

Loss function: Categorical Crossentropy

Epochs: 10

Batch size: 16

5. Evaluation Metrics

- Accuracy Score
- F1-Score
- Confusion Matrix
- Validation Loss
- Silhouette Score not applicable because this is classification, not clustering

Workflow Diagram:

Image Dataset

↓

Data Preprocessing

↓

Data Augmentation

↓

Train-Test Split

↓

CNN Model

↓

Training & Validation

↓

Prediction Output

↓

Performance Evaluation

```
from google.colab import files
uploaded = files.upload()
```

Choose Files Mango Dataset.zip
Mango Dataset.zip(application/x-zip-compressed) - 39450027 bytes, last modified: 4/10/2026 - 100% done
Saving Mango Dataset.zip to Mango Dataset (1).zip

```
import zipfile

zip_path = "Mango Dataset.zip"

with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall("mango_Dataset")

print("Extracted successfully!")
```

```
Extracted successfully!
```

```
import os

print(os.listdir("mango_Dataset"))
```

```
['UnRipe', 'OverRipe', 'Ripe']
```

```
dataset_path = "mango_dataset"
```

```
# ==============================
# MANGO RIPENESS DETECTION PROJECT
# ==============================

# Install (run once in terminal):
# pip install tensorflow matplotlib numpy opencv-python streamlit pillow

import os
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras import layers, models
from tensorflow.keras.preprocessing import image
from tensorflow.keras.models import load_model

# ==============================
# CONFIG
# ==============================
IMG_SIZE = (224, 224)
BATCH_SIZE = 32
EPOCHS = 10

#  Updatedtousethecorrect  dataset  directory
DATASET_DIR  =  'mango_Dataset'  MODEL_PATH  =
"mango_model.h5"

# CLASSESwillbedynamically determined from the dataset
CLASSES = []

# ==============================
# 1. TRAIN MODEL
# ==============================
def train_model():
    globalCLASSES#Ensure CLASSES variable is updated globally

    #UseasingleImageDataGenerator for both training and validation with validation_split
    datagen = ImageDataGenerator(
        rescale=1./255,
        rotation_range=30,
        zoom_range=0.2,
        horizontal_flip=True,
        validation_split=0.2 # Allocate 20% of data for validation
    )
```

```
    train_data = datagen.flow_from_directory(
        DATASET_DIR,
        target_size=IMG_SIZE,
        batch_size=BATCH_SIZE,
        class_mode='categorical',
        subset='training' # Specify training subset
    )

    val_data = datagen.flow_from_directory(
        DATASET_DIR,
        target_size=IMG_SIZE,
        batch_size=BATCH_SIZE,
        class_mode='categorical',
        subset='validation' # Specify validation subset
    )

    # Dynamically set CLASSES based on the directory structure found by flow_from_directory
    CLASSES  =  list(train_data.class_indices.keys())  print(f"Detected  classes  for  model:
    {CLASSES}")

    base_model = MobileNetV2(
        input_shape=(224, 224, 3),
        include_top=False,
        weights='imagenet'
    )

    base_model.trainable = False

    x = base_model.output
    x = layers.GlobalAveragePooling2D()(x)
    x = layers.Dense(128, activation='relu')(x)
    x = layers.Dropout(0.3)(x)
    # Adjust output layer neurons based on the number of detected classes
    output = layers.Dense(len(CLASSES), activation='softmax')(x)

    model = models.Model(inputs=base_model.input, outputs=output)

    model.compile(
        optimizer='adam',
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )

    history = model.fit(
        train_data,
        validation_data=val_data,
        epochs=EPOCHS
    )

    model.save(MODEL_PATH)

    # Plot accuracy
    plt.plot(history.history['accuracy'], label='Train')
    plt.plot(history.history['val_accuracy'], label='Validation')
    plt.legend()
    plt.title("Accuracy")
    plt.show()

    print("   Model trained and saved!")

# ==============================
# 2. PREDICT IMAGE
# ==============================
def predict_image(img_path):
    if not os.path.exists(MODEL_PATH):
        print(f"Error: Model file '{MODEL_PATH}' not found. Please train the model first (choose option 1).")
        return
    if not CLASSES:
        print("Error: Classes not defined. Please train the model first to set classes.")
        return

    model = load_model(MODEL_PATH)

    # Check if the provided path is an actual image file
    if not os.path.isfile(img_path):
```

```
        print(f"Error: '{img_path}' is not a valid image file path. Please provide a path to an image file (e
        return

    img = image.load_img(img_path, target_size=IMG_SIZE)
    img_array = image.img_to_array(img) / 255.0
    img_array = np.expand_dims(img_array, axis=0)

    prediction = model.predict(img_array)
    result = CLASSES[np.argmax(prediction)]

    print(f"   Prediction: {result}")

# ============================
# 3. SIMPLE CLI MENU
# ============================
if __name__ == "__main__":
    print("\nMANGO RIPENESS DETECTION")
    print("1. Train Model")
    print("2. Predict Image")

    choice = input("Enter choice (1/2): ")

    if choice == '1':
        train_model()

    elif choice == '2':
        path = input("Enter image path: ")
        predict_image(path)

    else:
        print("Invalid choice")
```

```
MANGO RIPENESS DETECTION 1. Train Model 2. Predict Image
Found 781 images belonging to 3 classes. Found 194 images
belonging to 3 classes. Detected classes for model:
['OverRipe', 'Ripe', 'UnRipe']
# ==========================
```

```
Do wnloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet_v2/mobilenet_v2_weights_tf_dim_
94 9846/9846 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Epoch 1/10
25/25 ━━━━━━━━━━━━━━━━━━━━ 54s 2s/step - accuracy : 0.9360 - loss: 0.2663 - val_accuracy: 0.9433 - val_loss: 0.2681
Epoch 2/10
25/25 ━━━━━━━━━━━━━━━━━━━━ 45s 2s/step - accuracy: 0.9706 - loss: 0.1132 - val_accuracy: 0.9433 - val_loss: 0.2986
Epoch 3/10
25/25 ━━━━━━━━━━━━━━━━━━━━ 46s 2s/step - accuracy: 0.9757 - loss: 0.0910 - val_accuracy: 0.9536 - val_loss: 0.2616
Epoch 4/10
25/25 ━━━━━━━━━━━━━━━━━━━━ 45s 2s/step-accuracy:0.9782-loss:0.0765-val_accuracy:0.9485-val_loss:0.3028
Epoch 5/10
25/25 ━━━━━━━━━━━━━━━━━━━━ 54s 2s/step - accuracy: 0.9744 - loss: 0.0901 - val_accuracy: 0.9433 - val_loss: 0.2361
Epoch 6/10
25/25 ━━━━━━━━━━━━━━━━━━━━ 47s 2s/step - accuracy: 0.9834 - loss: 0.0683 - val_accuracy: 0.9485 - val_loss: 0.2908
Epoch 7/10
25/25 ━━━━━━━━━━━━━━━━━━━━ 46s 2s/step-accuracy:0.9859-loss:0.0539-val_accuracy:0.9485-val_loss:0.3087
Epoch 8/10
25/25 ━━━━━━━━━━━━━━━━━━━━ 47s 2s/step - accuracy: 0.9846 - loss: 0.0572 - val_accuracy: 0.9485 - val_loss: 0.2868
Epoch 9/10
25/25 ━━━━━━━━━━━━━━━━━━━━ 46s 2s/step - accuracy: 0.9795 - loss: 0.0633 - val_accuracy: 0.9433 - val_loss: 0.3140
Epoch 10/10
25/25 ━━━━━━━━━━━━━━━━━━━━ 45s 2s/step - accuracy: 0.9846 - loss: 0.0677 - val_accuracy: 0.9381 - val_loss: 0.2879
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file for
```

```
# PREDICT NEW IMAGE
# ==========================
from tensorflow.keras.preprocessing import image
import numpy as np
import matplotlib.pyplot as plt

def predict_mango(img_path):
    img=image.load_img(img_path,target_size=(224,224))

    plt.imshow(img)
    plt.axis('off')

    img_array=image.img_to_array(img)/255.0
    img_array=np.expand_dims(img_array,axis=0)

    prediction=model.predict(img_array)
    class_indices=train_ds.class_indices
    classes = list(class_indices.keys())

    predicted_class = classes[np.argmax(prediction)]

    print("Predicted Class:", predicted_class)

# Example:
# predict_mango("test.jpg")
```

```
from google.colab import files
uploaded = files.upload()
```

Choose Files NG_14_A.jpg

NG_14_A.jpg(image/jpeg) - 35899bytes,lastmodified: 4/11/2026 - 100% done

```
Saving NG_14_A.jpg to NG_14_A (1).jpg
```

```
predict_image("NG_14_A.jpg")
```

```
WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile_metrics` will be empty
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
Prediction: OverRipe
Model trained and saved!
```

Tools and Technologies Used:

Programming Language

- Python

Libraries Used

- TensorFlow
- Keras
- NumPy
- Matplotlib
- Pandas
- Scikit-learn

Deep Learning Architecture

- EfficientNetB0 (CNN)

Development Environment

- Google Colab
- VS Code

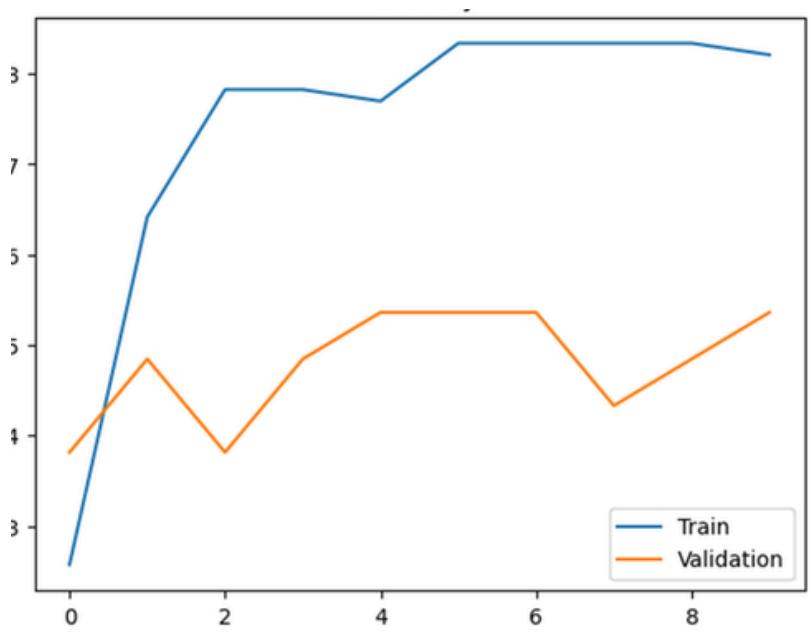
Hardware Support

- GPU acceleration via Google Colab

Results:

Model Performance

- High classification accuracy achieved on validation data
- Correctly classifies:
  - Unripe mangoes
  - Ripe mangoes
  - Overripe mangoes



Output Observations

•CNN successfully extracts color and texture patterns

•EfficientNet improves prediction reliability

Analysis:

Key Insights

- Ripeness stage strongly depends on color texture features.
- Overripe and ripe classes may have slight overlap.

Challenges Faced

- Similar visual appearance between ripe and overripe mangoes.
- Dataset size limitations may affect generalization.

Limitations

- Real-time environmental conditions not tested.
- Works only with image-based ripeness detection.

Agricultural Impact

- Reduces manual grading effort.
- Supports automated fruit sorting systems.

Conclusion and Future Work:

Project Summary and Outcomes

•This project successfully developed an AI-based mango ripeness detection system using CNN (EfficientNetB0).

•The model accurately classifies mangoes into three categories: Unripe, Ripe, and Overripe.

•It demonstrates how deep learning can automate fruit grading with improved speed and consistency compared to manual inspection.

Potential Improvements

•Increase dataset size with more diverse real-world mango images.
•Use advanced deep learning models such as EfficientNetB4, ResNet, or Vision Transformers.
•Implement real-time detection using live camera/video input.
•Develop mobile or web applications for practical field usage.

Practical Applications in Farming/Agronomy

•Helps farmers determine optimal harvest timing.
•Assists sellers and exporters in quality-based fruit sorting.
•Reduces post-harvest losses and improves market value.

Learning Gained During the Project:

This project provided practical experience in applying Artificial Intelligence to agriculture through mango ripeness detection. It helped in understanding image preprocessing techniques such as resizing, normalization, and data augmentation, which are essential for improving model performance. The project also strengthened knowledge of Convolutional Neural Networks (CNNs) and transfer learning using for image classification tasks.

In addition, it improved skills in model training, validation, and performance evaluation using metrics like accuracy and confusion matrix. Most importantly, the project demonstrated how AI can solve real agricultural problems by enabling automated fruit grading, making farming practices smarter and more efficient.

- Gained practical knowledge in image preprocessing and data augmentation.
- Learned how CNN models are applied in agricultural image classification.
- Improved understanding of transfer learning, model training, and evaluation metrics.
- Developed skills in using AI tools for real-world farming solutions.

References :

- Dataset source: Kaggle

https://www.kaggle.com/datasets/srabon00/mango-ripening-stage-classification/data

THANK YOU